

EXP-15

Aim

To write a Pandas program that retains only those rows in a DataFrame which contain at least two missing (NaN) values.

Algorithm

1. Import the Pandas library.
2. Create a DataFrame containing missing (NaN) values.
3. Use the isnull() function to identify missing values.
4. Count the number of NaN values in each row using sum(axis=1).
5. Filter the rows where the count of NaN values is greater than or equal to 2.
6. Display the resulting DataFrame.

Python Code

```
import pandas as pd
import numpy as np

# Create DataFrame
df = pd.DataFrame({
    'ord_no': [np.nan, np.nan, 70002, np.nan, np.nan, 70005, np.nan, 70010, 70003, 70012, np.nan, np.nan],
    'purch_amt': [np.nan, 270.65, 65.26, np.nan, 948.50, 2400.60, 5760.00, 1983.43, 2480.40, 250.45, 75.29, np.nan],
    'ord_date': [np.nan, '2012-09-10', np.nan, np.nan, '2012-09-10', '2012-07-27', '2012-09-10', '2012-10-10', '2012-10-10', '2012-06-27', '2012-08-17', np.nan],
    'customer_id': [np.nan, 3001, 3001, np.nan, 3002, 3001, 3001, 3004, 3003, 3002, 3001, np.nan]
})

print("Original DataFrame:\n")
print(df)

# Keep rows with at least 2 NaN values
result = df[df.isnull().sum(axis=1) >= 2]

print("\nRows with at least 2 NaN values:\n")
print(result)
```

Output

Original DataFrame:

	ord_no	purch_amt	ord_date	customer_id
0	NaN	NaN	NaN	NaN
1	NaN	270.65	2012-09-10	3001.0
2	70002.0	65.26	NaN	3001.0
3	NaN	NaN	NaN	NaN
4	NaN	948.50	2012-09-10	3002.0
5	70005.0	2400.60	2012-07-27	3001.0
6	NaN	5760.00	2012-09-10	3001.0
7	70010.0	1983.43	2012-10-10	3004.0
8	70003.0	2480.40	2012-10-10	3003.0
9	70012.0	250.45	2012-06-27	3002.0
10	NaN	75.29	2012-08-17	3001.0
11	NaN	NaN	NaN	NaN

Rows with at least 2 NaN values:

	ord_no	purch_amt	ord_date	customer_id
0	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN
11	NaN	NaN	NaN	NaN

Result

The program successfully filters and displays only those rows in the DataFrame that contain two or more missing (NaN) values using Pandas.

EXP 16

Aim

To write a Pandas program to split a DataFrame into groups based on school code and check the type of the GroupBy object.

Algorithm

1. Import the Pandas library.
2. Create a DataFrame with student details.
3. Use the groupby() function to group the data based on the school column.
4. Display each group.
5. Display the type of the GroupBy object using the type() function.

Python Code

```
import pandas as pd

# Create DataFrame
data = {
    'school': ['s001', 's002', 's003', 's001', 's002', 's004'],
    'class': ['V', 'V', 'VI', 'VI', 'V', 'VI'],
    'name': ['Alberto Franco', 'Gino Mcneill', 'Ryan Parkes',
            'Eesha Hinton', 'Gino Mcneill', 'David Parkes'],
    'date_of_birth': ['15/05/2002', '17/05/2002', '16/02/1999',
                     '25/09/1998', '11/05/2002', '15/09/1997'],
    'age': [12, 12, 13, 13, 14, 12],
    'height': [173, 192, 186, 167, 151, 159],
    'weight': [35, 32, 33, 30, 31, 32],
    'address': ['street1', 'street2', 'street3',
               'street1', 'street2', 'street4']
}

df = pd.DataFrame(data)

print("Original DataFrame:\n")
print(df)

# Group by school
grouped = df.groupby("school")

print("\nType of GroupBy Object:")
print(type(grouped))

print("\nGroups:\n")
for name, group in grouped:
    print("School:", name)
    print(group)
    print()
```

Output

Original DataFrame:

	school	class	name	date_Of_Birth	age	height	weight	address
0	s001	V	Alberto Franco	15/05/2002	12	173	35	street1
1	s002	V	Gino Mcneill	17/05/2002	12	192	32	street2
2	s003	VI	Ryan Parkes	16/02/1999	13	186	33	street3
3	s001	VI	Eesha Hinton	25/09/1998	13	167	30	street1
4	s002	V	Gino Mcneill	11/05/2002	14	151	31	street2
5	s004	VI	David Parkes	15/09/1997	12	159	32	street4

Type of GroupBy Object:

```
<class 'pandas.api.typing.DataFrameGroupBy'>
```

Groups:

School: s001

	school	class	name	date_Of_Birth	age	height	weight	address
0	s001	V	Alberto Franco	15/05/2002	12	173	35	street1
3	s001	VI	Eesha Hinton	25/09/1998	13	167	30	street1

School: s002

	school	class	name	date_Of_Birth	age	height	weight	address
1	s002	V	Gino Mcneill	17/05/2002	12	192	32	street2
4	s002	V	Gino Mcneill	11/05/2002	14	151	31	street2

School: s003

	school	class	name	date_Of_Birth	age	height	weight	address
2	s003	VI	Ryan Parkes	16/02/1999	13	186	33	street3

School: s004

	school	class	name	date_Of_Birth	age	height	weight	address
5	s004	VI	David Parkes	15/09/1997	12	159	32	street4

Result

The program successfully groups the DataFrame based on the school column and displays the type of the GroupBy object as DataFrameGroupBy.

Exp 17

Aim

To write a Pandas program to group the DataFrame by school code and calculate the mean, minimum, and maximum age of students in each school.

Algorithm

1. Import the Pandas library.
2. Create a DataFrame containing student details.
3. Group the DataFrame using the school column.
4. Select the age column.
5. Calculate the mean, minimum, and maximum age using the agg() function.
6. Display the results.

Python Code

```
import pandas as pd

# Create DataFrame
data = {
    'school': ['s001', 's002', 's003', 's001', 's002', 's004'],
    'class': ['V', 'V', 'VI', 'VI', 'V', 'VI'],
    'name': ['Alberto Franco', 'Gino Mcneill', 'Ryan Parkes',
            'Eesha Hinton', 'Gino Mcneill', 'David Parkes'],
    'date_of_Birth': ['15/05/2002', '17/05/2002', '16/02/1999',
                     '25/09/1998', '11/05/2002', '15/09/1997'],
    'age': [12, 12, 13, 13, 14, 12],
    'height': [173, 192, 186, 167, 151, 159],
    'weight': [35, 32, 33, 30, 31, 32],
    'address': ['street1', 'street2', 'street3',
               'street1', 'street2', 'street4']
}

df = pd.DataFrame(data)

# Group by school and calculate statistics
result = df.groupby("school")["age"].agg(["mean", "min", "max"])

print(result)
```

Output

```
      mean  min  max
school
s001    12.5   12   13
s002    13.0   12   14
s003    13.0   13   13
s004    12.0   12   12
|
```

Result

The program successfully groups the student records based on school code and calculates the mean, minimum, and maximum age for each school using the `groupby()` and `agg()` functions in Pandas.